

Roadmap — Sistema de Cine

Documento vivo. Se actualiza sesión a sesión a medida que se completan fases o cambian decisiones. La fuente de verdad del análisis funcional (requisitos, casos de uso, modelo de datos) es externa a este documento; acá se traduce ese análisis en un plan de ejecución ordenado.

ESTRUCTURA DE CARPETAS

```
src/app/
  core/           # SupabaseService, guards, interceptors, modelos TS compartidos
  shared/         # UI primitives propias (botón, card, badge, spinner, modal...)
  layout/         # header, footer, shell con outlet
  features/
    catalogo/     # M02 - listado, detalle, búsqueda/filtro, reseñas
    salas-butacas/ # M03/M04 - mapa de butacas, selección, disponibilidad
    compra/       # M05/M06/M07 - carrito, candy/combos, cupones, pago mock
  (NgModule)
    fidelizacion/ # M08 - puntos, crédito, canjes
    entradas/     # M09 - ticket PDF + QR (vista cliente)
    proximamente/ # M11 - estrenos, alertas
    perfil/       # M01 - registro, login, datos del cliente, Mis películas,
  cancelación (M10)
    empleado/     # validación QR/código manual
    administracion/ # M12/M13 + ABM de todo lo anterior (NgModule)
  app.routes.ts   # composición de rutas lazy por feature
```

ROADMAP POR FASES

Leyenda de estado:  Hecho ·  En progreso ·  Pendiente

Fase 0 — Fundaciones del proyecto

#	SUB-TAREA	ESTADO
0.1	Limpieza de repo y documentación base (README + CHANGELOG)	
0.2	Estructura de carpetas (<code>features/*</code> + rutas lazy placeholder)	
0.3	Supabase: cliente, <code>environment.ts</code> , <code>SupabaseService</code>	
0.4	Design system base: variables SCSS + componentes <code>Button / Card</code>	
0.5	Shell de layout: header, footer, fondo temático	

#	SUB-TAREA	ESTADO
0.6	Esquema SQL inicial en Supabase (tablas base + RLS)	✓
0.7	Seed de datos de demo	✓

Verificación 0.2 (hecha): `ng build` genera un chunk lazy por feature, `ng serve` levanta la app y navega entre las 10 páginas placeholder vía la navegación provisoria. `core/ / shared/ / layout/` se crean recién en 0.3/0.4/0.5, cuando tengan contenido real (git no versiona carpetas vacías).

Fase 1 — Catálogo público (M02) 🔄

Listado de películas (poster, nombre, duración, clasificación), detalle (sinopsis, géneros, reseñas con estrellas y promedio, funciones disponibles), buscador + filtro múltiple por género, destacado "3 más vendidas" en home.

#	SUB-TAREA	ESTADO
1.1	Capa de datos: modelos + <code>PeliculasService</code> (listado/detalle/destacadas) + destacado de "más vendidas" (contador <code>peliculas.entradas_vendidas</code>)	✓
1.2	Página de listado (home): grilla de películas + destacado "3 más vendidas"	✓
1.3	Buscador + filtro múltiple por género	✓
1.4	Página de detalle de película (sinopsis, géneros, funciones disponibles, reseñas + promedio)	■

Fase 2 — Salas, butacas y funciones (M03/M04) 🔄

Mapa visual de butacas (normal/accesible/VIP con diferenciación de color clara), disponibilidad en tiempo real, selección de función desde el detalle.

#	SUB-TAREA	ESTADO
2.1	Capa de datos: modelos (<code>Sala</code> , <code>Butaca</code> , <code>Funcion</code> , <code>ReservaButaca</code>) + <code>FuncionesService</code> (funciones disponibles por película)	✓
2.2	Selección de función desde el detalle de película (fecha, horario, sala, tipo de proyección 2D/3D/4D/5D, idioma)	✓
2.3	Mapa visual de butacas: layout por sala (filas/columnas) con diferenciación de color normal/accesible/VIP	✓
2.4	Bloqueo temporal de butacas en selección: alta/baja en <code>reservas_butaca</code> desde el frontend + expiración por <code>expira_at</code> — diferida	■
2.5	Disponibilidad en tiempo real (Supabase Realtime) reflejada en el mapa mientras otro usuario selecciona — diferida , se retoma después de 2.4	■
2.6	Selección de butacas en el mapa + confirmación → entrega el estado al carrito de la Fase 4	✓
2.7	Butacas ya vendidas bloqueadas en el mapa: política RLS de lectura pública sobre los <code>venta_items</code> de entrada no cancelados + estado "vendida" no seleccionable	✓

2.4 y 2.5 diferidas. Se adelanta 2.6 para cerrar el flujo película → función → butacas sin esperar el bloqueo en Postgres ni Realtime. Mientras tanto la selección es solo del lado del cliente (no bloquea nada). La confirmación de compra (4.6) se hizo sin esperar a la 2.4: el índice único `ux_butaca_por_funcion` impide vender dos veces la misma butaca, y un conflicto aparece recién al pagar. 2.4 se enchufa en `SeleccionButacasService.confirmar()` sin tocar el mapa.

2.7 agregada durante el desarrollo. Ninguna sub-tarea cubría las butacas *ya vendidas*: 2.4 y 2.5 tratan solo las que están en proceso de selección. El mapa ahora muestra como no seleccionables las butacas con una entrada vendida para esa función, con el mismo criterio que el índice `ux_butaca_por_funcion`.

Fase 3 — Autenticación y perfil (M01)

Registro (mail, nombre, apellido, fecha nacimiento, tipo de sangre, color de ojos, vacaciones anuales), login/logout, perfil (datos, puntos, crédito, historiales) — sin romper la navegación de invitado.

#	SUB-TAREA	ESTADO
3.1	Formulario de registro (alta en <code>perfiles</code> desde el frontend: nombre, apellido, fecha de nacimiento, tipo de sangre, color de ojos, vacaciones anuales) + <code>AuthService</code>	✓
3.2	Login/logout, persistencia de sesión y guard de rutas de invitado (ingreso/registro)	✓
3.3	Página de perfil: ver/editar datos propios (sin poder tocar <code>rol</code> /saldos, protegido también por trigger en backend)	✓
3.4	Perfil: saldo de puntos y crédito + historial de movimientos (<code>movimientos_puntos</code> / <code>movimientos_credito</code>)	✓
3.5	Perfil: historial de compras propias (<code>ventas</code>) y accesos rápidos ("Mis películas")	■
3.6	Navegación mixta invitado/registrado: header refleja sesión sin romper el flujo de compra anónima existente	■

Guards de sesión. En 3.2 se implementó `sinSessionGuard` (saca de ingreso/registro a quien ya tiene sesión) y en 3.3 `conSessionGuard` (exige sesión, aplicado al perfil). La confirmación por mail está desactivada en Supabase: el registro deja la sesión iniciada.

Fase 4 — Compra: entradas + Candy/combos + cupones (M05/M06/M07)

Carrito (entradas + productos + combos), cupón, compra anónima vs registrada, validación de edad (RN04), pantalla de pago simulada, persistencia de venta.

#	SUB-TAREA	ESTADO
4.1	Estructura <code>NgModule</code> de la feature <code>compra</code> + <code>CarritoService</code> (signals: entradas seleccionadas — tomadas de <code>SeleccionButacasService</code> de la 2.6 —, productos, combos)	✓
4.2	Candy Bar: listado de productos/combos por categoría, cantidad, agregar/quitar del carrito	✓

#	SUB-TAREA	ESTADO
4.3	Aplicación de cupón: validación de tipo (<code>primera_compra</code> / <code>edad</code> / <code>general</code>), vigencia y porcentaje; aplicación automática de <code>primera_compra</code> / <code>edad</code> si el usuario cumple las condiciones	✓
4.4	Checkout: datos de contacto, compra anónima vs registrada, validación de edad (RN04, <code>fecha_nacimiento_comprador</code>)	✓
4.5	Checkout registrado: uso opcional de crédito/puntos disponibles como medio de pago parcial (1 punto = \$1, combinable con crédito)	✓
4.6	Pantalla de pago simulada + confirmación de compra desde el frontend (crea <code>ventas</code> / <code>venta_items</code> / <code>entradas</code> / <code>pagos</code> , descuenta stock de productos — también los que vienen dentro de combos, vía <code>combo_items</code> —, acredita puntos)	✓
4.7	Pantalla de confirmación de compra (resumen, entradas emitidas)	✓

Fase 5 — Entradas y QR (M09, vista cliente) ✓

PDF de entrada + QR, pantalla "Mis entradas".

#	SUB-TAREA	ESTADO
5.1	Componente de visualización de QR por entrada (<code>entradas.codigo_qr</code> , generado en el cliente con <code>qrcode</code>)	✓
5.2	Generación de PDF de entrada (función, sala, butaca, QR), con <code>jspdf</code> cargado bajo demanda	✓
5.3	Pantalla "Mis entradas": listado propio, separación próximas/pasadas, estado (<code>emitida</code> / <code>validada</code> / <code>cancelada</code>)	✓

Fase 6 — Panel de empleado: validación (M09, operación) ✓

Escaneo/ingreso manual de código, validación de entrada y retiro de Candy, invalidación de QR usado.

#	SUB-TAREA	ESTADO
6.1	Guard de rol <code>empleado</code> + layout mínimo del panel	✓
6.2	Escaneo de QR por cámara + ingreso manual de código como alternativa	✓
6.3	Validación de entrada: marca <code>validada</code> , registra en <code>usos_qr</code> , rechaza código inexistente/ya usado	✓
6.4	Retiro de Candy: marca <code>ventas.candy_entregado_at</code> , independiente de la validación de entradas	✓
6.5	Historial de validaciones de la sesión del empleado en curso	✓

Una pantalla por concepto, consumo condicionado en Postgres. Todo quedó en `/empleado` (`ControlAcceso`), para empleado y admin: se elige ingreso a sala o retiro de Candy, se escanea con la cámara (`jsQR`) o se escribe el código, y se confirma después de ver la ficha. Cada consumo es un `update` condicionado al estado y cada intento queda en `usos_qr` ; el historial es el de los `usos_qr` propios desde el último ingreso. `candy_entregado_at` quedó a nivel de venta. Al probar las políticas se agregó un trigger que impide que un cliente complete o borre el retiro de Candy de su propia venta. También se extendió al empleado la regla de no operar como cliente (`noAdminGuard` pasó a `noPersonalGuard`), y el personal ya no ve Próximamente.

Fase 7 — Panel de administración: ABM base (M03/M04/M06/M07 admin) ✓

CRUD de películas, salas/butacas, funciones (asignación automática de sala + RN01/RN02/RN03), productos/categorías, combos, cupones y preventa.

#	SUB-TAREA	ESTADO
7.1	Estructura <code>NgModule</code> de la feature <code>administracion</code> + guard de rol <code>admin</code> + layout del panel	✓
7.2	ABM de películas y géneros (incluye <code>preventa_habilitada</code> / <code>precio_preventa</code>)	✓
7.3	ABM de salas y butacas (layout de filas/columnas, tipo normal/accesible/VIP)	✓
7.4	ABM de funciones: asignación automática de sala o elegida a mano, validación de solapamiento con margen de 30 min (RN01/RN02/RN03)	✓
7.5	ABM de productos/categorías y combos (con <code>combo_items</code>)	✓
7.6	ABM de cupones (tipo, vigencia, porcentaje)	✓

El admin no opera como cliente. Cerrada la Fase 7 se agregó que un admin puede recorrer el catálogo y ver las funciones, pero no comprar: el encabezado no le muestra "Mis entradas"/"Mi perfil" y `noAdminGuard` cierra `/butacas` , `/compra` , `/mis-entradas` y `/cuenta/perfil` .

Fase 8 — Fidelización y crédito (M08) ✓

Acreditación de puntos, configuración y canje de recompensas, saldo e historial, no transferencia.

#	SUB-TAREA	ESTADO
8.1	Acreditación de puntos al confirmar una compra (integra con la confirmación de compra de la Fase 4, vía <code>movimientos_puntos</code>)	✓
8.2	ABM de recompensas desde administración (<code>recompensas</code> : tipo entrada/producto, costo en puntos)	✓
8.3	Canje de recompensas en el checkout (<code>canjes</code> + débito de puntos, atado a la venta)	✓
8.4	Vista de saldo e historial unificado de puntos y crédito en el perfil	✓
8.5	Verificación de la regla "no transferencia": puntos/crédito solo se mueven por operaciones propias del usuario (políticas RLS + trigger de saldos), nunca por edición directa	✓

Canje dentro de la compra. La 8.3 se planeó como canje desde el perfil, pero se resolvió canjear durante la compra: la recompensa de entrada cubre la entrada de menor precio y la de producto lo suma sin cargo. El perfil muestra el catálogo de recompensas y el historial de canjes. La 8.1 ya estaba hecha desde la Fase 4.6. No se creó la carpeta `features/fidelizacion/`: el servicio de recompensas vive en `core/` y los componentes en `compra/`, `perfil/` y `administracion/`.

Fase 9 — Cancelaciones (M10) ✓

Cancelación hasta 2 horas antes, liberación de butacas, generación de crédito.

#	SUB-TAREA	ESTADO
9.1	Cancelación de compra desde "Mis entradas"/perfil, con validación de ventana (hasta 2 hs antes de la función)	✓
9.2	Cancelación: marca <code>ventas / venta_items</code> cancelados (libera el índice único de butaca), genera <code>movimientos_credito</code> , actualiza <code>peliculas.entradas_vendidas</code>	✓
9.3	Cancelación desde el panel de administración (con motivo, sin restricción de ventana horaria)	✓

Cancelación desde el perfil y crédito por trigger. El cliente cancela desde "Mis compras" del perfil (la cancelación es por venta, y "Mis entradas" agrupa por función). El frontend solo pasa la venta a `cancelada`; el trigger de cancelación cancela ítems y entradas, acredita `total - puntos_usados` como crédito y revierte el neto de puntos de la venta, así funciona igual cuando cancela el admin, que no puede escribir en ledgers ajenos. El motivo del admin se guarda en la nueva columna `ventas.motivo_cancelacion`. No se creó `features/cancelaciones/`.

Fase 10 — Próximamente y notificaciones (M11) ✓

Sección "Próximamente", alertas de estreno, preventa 7 días antes, notificación in-app al habilitarse la venta.

#	SUB-TAREA	ESTADO
10.1	Sección pública "Próximamente" (películas con preventa habilitada o estreno futuro)	✓
10.2	Suscripción a alertas de estreno desde el perfil (<code>alertas_estreno</code>)	✓
10.3	Preventa: habilitación de venta 7 días antes del estreno con <code>precio_preventa</code>	✓
10.4	Notificaciones in-app al habilitarse la preventa/venta (<code>notificaciones</code>) + indicador de no leídas	✓

Alertas desde Próximamente, avisos generados por el frontend. La alerta se activa desde Próximamente o desde el detalle de la película, y el perfil suma "Alertas de estreno" para ver las pendientes y quitarlas. La ventana de preventa se calcula en la aplicación a partir de `fecha_estreno`, sin columna ni job programado. Como la apertura de la venta depende del paso del tiempo, el aviso lo genera el frontend al abrir la app con sesión: inserta la notificación y marca la alerta como cumplida. Para eso se agregó `notificaciones.pelicula_id` y una política de insert propio. El catálogo pasa a mostrar solo películas estrenadas o en preventa.

Fase 11 — Reportes y estadísticas (M12)

Facturación diaria, entradas vendidas, exportación PDF/Excel, gráficos de películas más vistas y producto más vendido.

#	SUB-TAREA	ESTADO
11.1	Reporte de facturación por día/rango de fechas	
11.2	Reporte de entradas vendidas por película/función/periodo	
11.3	Gráficos: películas más vistas y producto más vendido	
11.4	Exportación de reportes a PDF/Excel	

Una sola página de reportes, agregada en el cliente. Las cuatro sub-tareas quedaron en `/administracion/reportes`: período por atajos (hoy, esta semana, este mes) o rango, y pestañas de facturación diaria, entradas y Candy Bar. Se consultan las ventas pagadas del período con sus ítems y se suman en el navegador, sin migración. "Más vistas" se mide por entradas vendidas (al cerrarla, la validación de la Fase 6 seguía pendiente) y el producto más vendido cuenta también los que vienen en combos. Los gráficos son barras propias en HTML/CSS y el Excel usa `write-excel-file`.

Fase 12 — Auditoría (M13)

Registro y consulta de logs de acciones administrativas y de validación.

#	SUB-TAREA	ESTADO
12.1	Registro en <code>logs_actividad</code> desde las acciones administrativas de las Fases 7-11 (desde los servicios o con triggers)	
12.2	Registro en <code>logs_actividad</code> de las validaciones de QR/Candy del panel de empleado (Fase 6)	
12.3	Pantalla de consulta de logs en administración, con filtros por usuario/entidad/acción/fecha	

Fase 13 — PWA y pulido final

Manifest + service worker, instalabilidad, revisión de RLS, accesibilidad, responsive final, despliegue a Vercel, README de arquitectura (RNF-008).

#	SUB-TAREA	ESTADO
13.1	Manifest + service worker + instalabilidad (Angular PWA)	☐
13.2	Revisión final de políticas RLS: cobertura de las 26 tablas y de cada rol (anon/authenticated/empleado/admin)	☐
13.3	Accesibilidad (a11y): contraste, foco, roles ARIA en componentes propios del design system	☐
13.4	Responsive final en todas las features	☐
13.5	Despliegue a Vercel + variables de entorno de producción	☐
13.6	README de arquitectura final (RNF-008)	☐

CONVENCIÓN TRANSVERSAL

- Cada PR agrega una entrada nueva en `CHANGELOG.md`.
- El `README.md` se actualiza cuando una fase introduce una decisión de arquitectura nueva.
- Cada PR describe: Objetivo inicial / Qué se terminó haciendo / Resumen de cambios. Sin líneas de atribución al agente.